



LambdaProof Security Test Suite Specification

Adversarial Testing Framework

Version: 1.0

Effective Date: January 8, 2026

Repository: github.com/lproof/security-tests

License: Apache 2.0

Abstract

This specification defines the adversarial testing framework for LambdaProof implementations, establishing attack scenarios, red team methodologies, fuzzing protocols, and security validation requirements. The Security Test Suite (SEC) extends the Conformance Test Suite to validate that implementations resist active adversaries, not merely function correctly under benign conditions. This framework is essential for Tier 2 (Proof-Verified) and Tier 3 (Proof-Certified Enterprise) certification.^[1]

Section 1: Scope and Objectives

1.1 Security Testing Goals

Goal	Description
Attack Resistance	Validate implementations resist known attack classes
Fail-Closed Verification	Confirm systems block unauthorized actions under adversarial conditions
CSL Invariant Preservation	Ensure ethical operators cannot be bypassed or manipulated
Audit Integrity	Verify Δ -Archivum remains tamper-evident under attack
Silence Clause Enforcement	Confirm uncertain states trigger NO-OP, not default-allow

1.2 Testing Categories

```

lproof-security-tests/
├─ attack-vectors/
│   ├── token-attacks/           # Capability token manipulation
│   ├── csl-bypass/             # CSL operator circumvention
│   ├── prime-gate-attacks/      # Prime-gated activation exploits
│   ├── audit-tampering/        #  $\Delta$ -Archivum integrity attacks
│   ├── adapter-bypass/         # Enforcement boundary attacks
│   └─ cryptographic/           # Cryptographic implementation flaws
├─ fuzzing/
│   ├── intent-fuzzer/          # Intent schema fuzzing
│   ├── token-fuzzer/           # Token structure fuzzing
│   └─ policy-fuzzer/           # Policy rule fuzzing
├─ red-team/
│   ├── scenarios/              # Multi-step attack scenarios
│   ├── campaigns/              # Extended adversarial campaigns
│   └─ reports/                 # Attack outcome documentation
├─ regression/
│   ├── cve-reproductions/       # Known vulnerability reproductions
│   └─ incident-tests/          # Tests from past incidents
└─ harness/
    ├── adversary-simulator.ts   # Adversarial action generator
    ├── attack-reporter.ts       # Security finding reports
    └─ coverage-tracker.ts       # Attack surface coverage

```

Section 2: Attack Vector Categories

2.1 Category SEC-TOK: Token Attacks

Tests targeting capability token manipulation, forgery, and misuse.^{[2][1]}

Test ID	Name	Attack Vector	Expected Defense
SEC-TOK-001	Token Signature Forgery	Submit token with invalid Ed25519 signature	Signature verification failure, BLOCKED
SEC-TOK-002	Token Replay Attack	Resubmit previously-used single-use token	Replay protection triggers, BLOCKED
SEC-TOK-003	Token Expiry Bypass	Submit token with manipulated notAfter	Server-side expiry check fails, BLOCKED
SEC-TOK-004	Intent Hash Mismatch	Token with valid sig but mismatched intent	Binding validation fails, BLOCKED
SEC-TOK-005	Use Count Manipulation	Decrement usedCount in tampered token	Signature invalidation, BLOCKED
SEC-TOK-006	Scope Escalation	Expand constraints beyond original intent	Intent hash mismatch, BLOCKED
SEC-TOK-007	Cross-Session Token Use	Use token from session A in session B	Session binding fails, BLOCKED
SEC-TOK-008	Token Theft Replay	Valid token from compromised session	Drift detection triggers, SILENT
SEC-TOK-009	Issuer Impersonation	Token signed by non-authorized key	Key verification fails, BLOCKED
SEC-TOK-010	Token Timing Attack	Exploit timing differences in verification	Constant-time comparison enforced

2.2 Category SEC-CSL: CSL Bypass Attacks

Tests attempting to circumvent Conscious Sovereignty Layer operators.^[1]

Test ID	Name	Attack Vector	Expected Defense
SEC-CSL-001	Neutrality Evasion	Craft intent that appears neutral but discriminates	Deep neutrality check fails, SILENT
SEC-CSL-002	Beneficence False Positive	Intent extracts value while appearing beneficial	Beneficence operator catches, SILENT
SEC-CSL-003	Silence Override Attempt	Force action when CSL returns UNCERTAIN	Silence Clause enforced, NO-OP
SEC-CSL-004	Operator Order Manipulation	Exploit non-commuting operator sequences	Commutation invariant enforced
SEC-CSL-005	Partial CSL Satisfaction	Pass N, fail B, attempt execution	All operators required, SILENT
SEC-CSL-006	CSL State Confusion	Inject inconsistent CSL evaluation context	Context validation fails, SILENT
SEC-CSL-007	Ethics Guardian Bypass	Attempt governance action bypassing Ethics Guardian	Veto authority preserved
SEC-CSL-008	Silence Clause Race	Concurrent intents exploiting evaluation timing	Serialized evaluation enforced

2.3 Category SEC-GATE: Prime Gate Attacks

Tests targeting prime-gated activation mechanisms.^{[3][1]}

Test ID	Name	Attack Vector	Expected Defense
SEC-GATE-001	Composite Epoch Injection	Submit at composite epoch claiming prime	Prime verification fails, SILENT
SEC-GATE-002	Prime Prediction	Pre-compute tokens for future prime epochs	Epoch binding in signature
SEC-GATE-003	Miller-Rabin Spoofing	Submit composite passing insufficient rounds	$k \geq 40$ rounds enforced

SEC-GATE-004	Epoch Boundary Race	Submit at exact epoch transition boundary	Deterministic epoch resolution
SEC-GATE-005	Prime Salt Collision	Generate colliding prime salts	Poseidon collision resistance
SEC-GATE-006	Epoch Rollback	Claim earlier epoch for replay	Monotonic epoch enforcement

2.4 Category SEC-ARC: Audit Tampering Attacks

Tests targeting Λ -Archivum integrity.^{[4][1]}

Test ID	Name	Attack Vector	Expected Defense
SEC-ARC-001	Hash Chain Break	Insert event breaking <code>prevEventHash</code> link	Chain verification fails
SEC-ARC-002	Event Deletion	Remove event from middle of chain	Gap detection triggers
SEC-ARC-003	Event Reordering	Swap timestamp order of events	Monotonic timestamp violation
SEC-ARC-004	Merkle Proof Forgery	Generate invalid inclusion proof	Proof verification fails
SEC-ARC-005	Checkpoint Manipulation	Alter Merkle root in checkpoint	Multi-chain anchor mismatch
SEC-ARC-006	PII Injection	Attempt to store PII in audit events	Schema validation rejects
SEC-ARC-007	Anchor Divergence	Submit conflicting anchors to different chains	Threshold anchoring detects
SEC-ARC-008	Selective Disclosure	Omit events from verification request	Merkle proof incomplete

2.5 Category SEC-ADAPT: Adapter Bypass Attacks

Tests targeting adapter enforcement boundaries.^[2]

Test ID	Name	Attack Vector	Expected Defense
SEC-ADAPT-001	Direct Tool Invocation	Bypass adapter, call tool directly	Tool requires adapter attestation
SEC-ADAPT-002	Adapter Impersonation	Forge adapter identity	Adapter key verification
SEC-ADAPT-003	Enforcement Mode Downgrade	Force audit-only mode in production	Mode locked at initialization
SEC-ADAPT-004	Silent Mode Escape	Execute during drift-exceeded silent mode	Silent mode is absolute
SEC-ADAPT-005	Parameter Injection	Inject params not in original intent	Intent hash binding
SEC-ADAPT-006	Adapter State Corruption	Manipulate adapter internal state	Stateless verification design
SEC-ADAPT-007	Multi-Adapter Confusion	Exploit routing between adapters	Single-adapter-per-intent binding

2.6 Category SEC-CRYPTO: Cryptographic Attacks

Tests targeting cryptographic implementations.^[4]

Test ID	Name	Attack Vector	Expected Defense
SEC-CRYPTO-001	Poseidon Preimage	Attempt hash preimage recovery	Cryptographic hardness
SEC-CRYPTO-002	Ed25519 Key Recovery	Extract signing key from signatures	Constant-time implementation
SEC-CRYPTO-003	SNARK Soundness	Forge ZK proof without witness	Groth16 soundness
SEC-CRYPTO-004	Randomness Weakness	Exploit weak nonce generation	CSPRNG enforcement
SEC-CRYPTO-005	Side Channel Timing	Extract secrets via timing analysis	Constant-time operations
SEC-CRYPTO-006	Hash Length Extension	Exploit SHA-256 construction	HMAC or Poseidon used

SEC-CRYPTO-007	Commitment Opening	Open commitment to different value	Binding property enforced
----------------	--------------------	------------------------------------	---------------------------

Section 3: Adversarial Test Patterns

3.1 Attack Test Structure

```
// src/attack-vectors/token-attacks/replay.test.ts

import { describe, it, expect } from '@lproof/test-harness';
import { AdversarySimulator } from '@lproof/adversary';
import { Implementation } from '@lproof/adapters';

/**
 * SEC-TOK-002: Token Replay Attack
 *
 * Attack Vector: Attacker captures valid single-use token,
 *                 attempts to replay after legitimate use.
 *
 * Threat Model: Network eavesdropper with token capture capability
 * Severity: CRITICAL
 * CVSS: 9.1 (Critical)
 * Constitution Reference: Article II, Silence Clause
 */
describe('Token Replay Attack', () => {
  let impl: Implementation;
  let adversary: AdversarySimulator;

  beforeAll(async () => {
    impl = await Implementation.connect();
    adversary = new AdversarySimulator(impl);
  });

  it('SEC-TOK-002: rejects replayed single-use token', async () => {
    // SETUP: Create legitimate token
    const token = await impl.createToken({
      intent: fixtures.validIntent,
```

```

    constraints: { singleUse: true }
  });

  // LEGITIMATE USE: First use succeeds
  const firstUse = await impl.enforceAction({
    token: token.token,
    action: fixtures.validAction
  });
  expect(firstUse.status).toBe('EXECUTED');

  // ATTACK: Replay the same token
  const replayAttempt = await adversary.replayToken({
    token: token.token,
    action: fixtures.validAction
  });

  // DEFENSE VERIFICATION
  expect(replayAttempt.status).toBe('BLOCKED');
  expect(replayAttempt.reason).toBe('TOKEN_USES_EXHAUSTED');

  // Verify audit trail captures attack
  const auditEvent = await impl.getAuditEvent(replayAttempt.auditEventId);
  expect(auditEvent.eventType).toBe('REPLAY_ATTEMPT_BLOCKED');
  expect(auditEvent.attackVector).toBe('TOKEN_REPLAY');
});

it('SEC-TOK-002b: detects replay across distributed nodes', async () => {
  // More sophisticated attack: race condition across nodes
  const token = await impl.createToken({
    intent: fixtures.validIntent,
    constraints: { singleUse: true }
  });

  // ATTACK: Simultaneous submission to multiple nodes
  const [result1, result2] = await Promise.all([
    adversary.submitToNode('node-1', token.token, fixtures.validAction),
    adversary.submitToNode('node-2', token.token, fixtures.validAction)
  ]);

  // DEFENSE: Exactly one succeeds, one fails

```

```

    const successes = [result1, result2].filter(r => r.status === 'EXECUTED');
    const failures = [result1, result2].filter(r => r.status === 'BLOCKED');

    expect(successes.length).toBe(1);
    expect(failures.length).toBe(1);
    expect(failures[0].reason).toBe('TOKEN_USES_EXHAUSTED');
  });
});

```

3.2 CSL Bypass Test Pattern

```

// src/attack-vectors/csl-bypass/silence-override.test.ts

/**
 * SEC-CSL-003: Silence Override Attempt
 *
 * Attack Vector: Attacker crafts intent that causes UNCERTAIN
 *                  CSL evaluation, then attempts to force execution.
 *
 * Threat Model: Malicious agent with intent crafting capability
 * Severity: CRITICAL
 * Constitution Reference: Article II, Section 2 - Silence Clause
 */
describe('Silence Override Attack', () => {
  it('SEC-CSL-003: cannot force action when CSL returns UNCERTAIN', async () => {
    // SETUP: Create intent that triggers uncertainty
    const ambiguousIntent = {
      tool: 'payments',
      action: 'transfer',
      params: {
        amount: 9999.99, // Edge case amount
        recipient: 'UNKNOWN_ENTITY',
        memo: 'Potentially beneficial OR harmful' // Ambiguous
      }
    };

    // CSL evaluation returns UNCERTAIN
    const cslResult = await impl.evaluateCSL(ambiguousIntent);
    expect(cslResult.neutrality).toBe('UNCERTAIN');
  });
});

```

```

expect(cslResult.beneficence).toBe('UNCERTAIN');

// ATTACK: Attempt to force token issuance despite uncertainty
const attackPayloads = [
  { forceIssuance: true },
  { bypassCSL: true },
  { assumeBeneficent: true },
  { defaultAllow: true }
];

for (const payload of attackPayloads) {
  const attackResult = await adversary.forceTokenIssuance({
    intent: ambiguousIntent,
    ...payload
  });

  // DEFENSE: Silence Clause MUST trigger
  expect(attackResult.tokenIssued).toBe(false);
  expect(attackResult.status).toBe('SILENT');
  expect(attackResult.auditEvent.type).toBe('CSL_SILENCE_TRIGGERED');
  expect(attackResult.auditEvent.reason).toBe('UNCERTAINTY_DETECTED');
}
});

it('SEC-CSL-003b: Silence persists across retry attempts', async () => {
  const ambiguousIntent = fixtures.createAmbiguousIntent();

  // ATTACK: Rapid retry hoping for different evaluation
  const attempts = await Promise.all(
    Array(100).fill(null).map(() =>
      adversary.attemptExecution(ambiguousIntent)
    )
  );

  // DEFENSE: All 100 attempts must be silenced
  const silenced = attempts.filter(a => a.status === 'SILENT');
  expect(silenced.length).toBe(100);
});
});

```

Section 4: Fuzzing Framework

4.1 Intent Schema Fuzzing

```
// src/fuzzing/intent-fuzzer/fuzzer.ts

interface FuzzConfig {
  seed: number;
  iterations: number;
  maxDepth: number;
  mutationRate: number;
  corpus: Intent[];
}

class IntentFuzzer {
  private rng: SeededRandom;
  private mutators: Mutator[];
  private corpus: Intent[];

  constructor(config: FuzzConfig) {
    this.rng = new SeededRandom(config.seed);
    this.mutators = [
      new FieldDeletionMutator(),
      new TypeConfusionMutator(),
      new BoundaryValueMutator(),
      new UnicodeInjectionMutator(),
      new NestedObjectMutator(),
      new ArrayOverflowMutator(),
      new NullInjectionMutator(),
      new SignExtensionMutator()
    ];
  }

  async fuzz(target: Implementation): Promise<FuzzReport> {
    const findings: Finding[] = [];

    for (let i = 0; i < this.config.iterations; i++) {
      const baseIntent = this.selectFromCorpus();
```

```

const mutatedIntent = this.mutate(baseIntent);

try {
  const result = await target.processIntent(mutatedIntent);

  // Check for unexpected behaviors
  if (this.isUnexpectedSuccess(result, mutatedIntent)) {
    findings.push({
      type: 'UNEXPECTED_SUCCESS',
      severity: 'HIGH',
      input: mutatedIntent,
      output: result
    });
  }
} catch (error) {
  if (this.isCrash(error)) {
    findings.push({
      type: 'CRASH',
      severity: 'CRITICAL',
      input: mutatedIntent,
      error: error.message
    });
  }
}

return { findings, coverage: this.getCoverage() };
}

```

4.2 Mutation Strategies

Mutator	Description	Target Vulnerabilities
FieldDeletion	Remove required fields	Schema validation bypass
TypeConfusion	Change field types (string→number)	Type coercion bugs
BoundaryValue	INT_MAX, INT_MIN, 0, -1	Integer overflow/underflow

UnicodeInjection	NULL bytes, RTL override, homoglyphs	Unicode handling bugs
NestedObject	Deep nesting (10,000+ levels)	Stack overflow, DoS
ArrayOverflow	Arrays with millions of elements	Memory exhaustion
NullInjection	Null/undefined in unexpected places	Null pointer dereference
SignExtension	Negative values in unsigned contexts	Sign extension bugs

Section 5: Red Team Scenarios

5.1 Multi-Stage Attack Scenarios

```
# red-team/scenarios/privilege-escalation.yaml
scenario:
  id: RT-001
  name: "Progressive Privilege Escalation"
  description: "Attacker starts with minimal access, attempts full system compromise"
  threat_actor: "Sophisticated external attacker"
  duration: "72 hours simulated"

  stages:
    - id: reconnaissance
      name: "Reconnaissance"
      duration: "4 hours"
      actions:
        - "Enumerate public API endpoints"
        - "Analyze SDK source code"
        - "Map intent schema structure"
        - "Identify adapter types"
      success_criteria:
        - "Documented API surface"
        - "Identified potential attack vectors"

    - id: initial_access
```

```
name: "Initial Access"
duration: "8 hours"
actions:
  - "Create legitimate low-privilege account"
  - "Obtain valid capability token for minimal scope"
  - "Test token boundaries"
success_criteria:
  - "Valid session established"
  - "Low-privilege token obtained"

- id: privilege_escalation
  name: "Privilege Escalation Attempts"
  duration: "24 hours"
  attacks:
    - vector: "Token scope expansion"
      method: "Modify constraints in token body"
      expected_defense: "Signature invalidation"

    - vector: "CSL bypass via edge case"
      method: "Craft intent at neutrality boundary"
      expected_defense: "Silence clause triggers"

    - vector: "Prime gate timing attack"
      method: "Pre-compute tokens for prime epochs"
      expected_defense: "Epoch binding in signature"

    - vector: "Adapter direct call"
      method: "Bypass policy engine, call adapter"
      expected_defense: "Adapter requires valid token"

- id: persistence
  name: "Persistence Attempts"
  duration: "12 hours"
  attacks:
    - vector: "Token stockpiling"
      method: "Generate many tokens for later use"
      expected_defense: "Expiry enforcement"

    - vector: "Session fixation"
      method: "Hijack another user's session"
```

```

    expected_defense: "Session-token binding"

- id: exfiltration
  name: "Data Exfiltration Attempts"
  duration: "24 hours"
  attacks:
    - vector: "Audit log PII extraction"
      method: "Query  $\Delta$ -Archivum for user data"
      expected_defense: "Hash-only semantics, no PII"

    - vector: "Intent parameter recovery"
      method: "Reverse canonical hash"
      expected_defense: "Cryptographic preimage resistance"

success_metrics:
  full_compromise: false
  partial_compromise: false
  information_disclosure: false
  denial_of_service: false

defenses_validated:
  - "Token cryptographic binding"
  - "CSL operator enforcement"
  - "Prime-gated activation"
  - "Fail-closed adapter enforcement"
  - "Audit log privacy preservation"

```

5.2 Insider Threat Scenario

```

# red-team/scenarios/insider-threat.yaml
scenario:
  id: RT-002
  name: "Malicious Insider Attack"
  description: "Certified developer attempts to introduce backdoor"
  threat_actor: "Trusted insider with commit access"

  attack_vectors:
    - name: "Backdoor in reference implementation"
      method: "Submit PR with subtle vulnerability"

```

```

defenses:
  - "Multi-reviewer code review"
  - "Automated security scanning"
  - "Conformance test regression"

- name: "Test vector manipulation"
  method: "Modify golden vectors to accept invalid inputs"
  defenses:
    - "Cryptographic signing of test vectors"
    - "Cross-implementation validation"

- name: "Documentation sabotage"
  method: "Document incorrect security properties"
  defenses:
    - "Specification/implementation correspondence"
    - "Community review process"

```

Section 6: Security Test Requirements by Certification Tier

6.1 Tier Requirements Matrix

Test Category	Tier 1 (Compatible)	Tier 2 (Verified)	Tier 3 (Certified)
SEC-TOK (Token)	100% pass	100% pass + audit	100% pass + pen test
SEC-CSL (CSL Bypass)	100% pass	100% pass + audit	100% pass + red team
SEC-GATE (Prime Gate)	100% pass	100% pass + audit	100% pass + pen test
SEC-ARC (Audit)	100% pass	100% pass + audit	100% pass + tamper test
SEC-ADAPT (Adapter)	100% pass	100% pass + audit	100% pass + bypass test
SEC-CRYPTO	100% pass	100% pass + audit	100% pass + crypto review
Fuzzing	10,000 iterations	100,000 iterations	1,000,000 iterations
Red Team	Not required	1 scenario	Full campaign

6.2 Tier 2 Security Audit Scope

Tier 2 Security Audit Requirements

Mandatory Review Areas

1. ****CSL Enforcement Correctness****
 - All three operators (N, B, S) correctly implemented
 - Commutation invariant preserved
 - Silence Clause triggers on uncertainty
2. ****Cryptographic Implementation****
 - Poseidon hash correctness
 - Ed25519 signature verification
 - Randomness quality (CSPRNG)
 - Constant-time operations
3. ****Token Security****
 - Binding validation
 - Expiry enforcement
 - Replay protection
 - Use count tracking
4. ****Audit Integrity****
 - Hash chaining correctness
 - Merkle checkpoint verification
 - No PII in events
 - Tamper detection
5. ****Privacy Preservation****
 - Zero-surveillance compliance
 - Data minimization
 - Consent enforcement

Audit Deliverables

- [] Security findings report (CVSS scored)
- [] Remediation verification
- [] Conformance test results
- [] Fuzzing coverage report

Section 7: Vulnerability Classification

7.1 Severity Levels

Severity	CVSS Range	Description	Required Response
CRITICAL	9.0-10.0	Complete system compromise, CSL bypass, or mass privacy breach	Immediate patch, emergency disclosure
HIGH	7.0-8.9	Significant security control bypass, partial system compromise	Patch within 7 days
MEDIUM	4.0-6.9	Limited impact, defense-in-depth failure	Patch within 30 days
LOW	0.1-3.9	Minor security weakness, theoretical attack	Patch in next release

7.2 Constitutional Violation Severity

Violation	Severity	Constitution Reference
Silence Clause bypass	CRITICAL	Article II, Section 2
CSL operator failure	CRITICAL	Article II, Section 2.2
Audit tampering	CRITICAL	Article V
Prime gate bypass	HIGH	Article III
PII disclosure in logs	HIGH	Article V, Section 5.2
Token replay success	HIGH	Protocol Specification
Drift bound violation	MEDIUM	Article III, Section 3.3

Section 8: Security Test Execution

8.1 Running Security Tests

```
# Install security test suite
npm install @lproof/security-tests

# Run all security tests
lproof-security test --implementation ./your-impl --output ./security-report.json

# Run specific attack category
lproof-security test --category SEC-CSL --implementation ./your-impl

# Run fuzzing campaign
lproof-security fuzz --target ./your-impl --iterations 100000 --seed 42

# Generate Tier 2 security report
lproof-security report --results ./security-report.json --tier 2 --output
./tier2-security-audit.pdf
```

8.2 CI/CD Integration

```
# .github/workflows/security-tests.yml
name: Security Test Suite

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
  schedule:
    - cron: '0 2 * * *' # Nightly security scan

jobs:
  security-tests:
    runs-on: ubuntu-latest
    steps:
```

```

- uses: actions/checkout@v4

- name: Run SEC-TOK Tests
  run: lproof-security test --category SEC-TOK

- name: Run SEC-CSL Tests
  run: lproof-security test --category SEC-CSL

- name: Run SEC-GATE Tests
  run: lproof-security test --category SEC-GATE

- name: Run Fuzzing (CI Mode)
  run: lproof-security fuzz --iterations 10000 --timeout 300

- name: Upload Security Report
  uses: actions/upload-artifact@v4
  with:
    name: security-report
    path: ./security-report.json

- name: Fail on Critical/High Findings
  run: |
    CRITICAL=$(jq '.findings | map(select(.severity == "CRITICAL")) | length'
security-report.json)
    HIGH=$(jq '.findings | map(select(.severity == "HIGH")) | length'
security-report.json)
    if [ "$CRITICAL" -gt 0 ] || [ "$HIGH" -gt 0 ]; then
      echo "Security tests found $CRITICAL critical and $HIGH high severity
issues"
      exit 1
    fi

```

Section 9: Responsible Disclosure

9.1 Vulnerability Reporting

Security vulnerabilities discovered through this test suite or independent research should be reported through the AProof Vulnerability Disclosure Program:

Reporting Channel: security@lambdaproof.io

PGP Key: Available at <https://lambdaproof.io/.well-known/security.txt>

Response SLA: Initial response within 24 hours

9.2 Disclosure Timeline

Phase	Timeline	Actions
Report received	T+0	Acknowledge receipt, assign severity
Triage complete	T+48h	Confirm vulnerability, assign CVE if warranted
Patch development	T+7d (Critical) / T+30d (High)	Develop and test fix
Coordinated disclosure	T+90d maximum	Public disclosure with patch available

Section 10: Prior Art Statement

This specification is published as prior art under CC0 1.0 Universal Public Domain Dedication to:

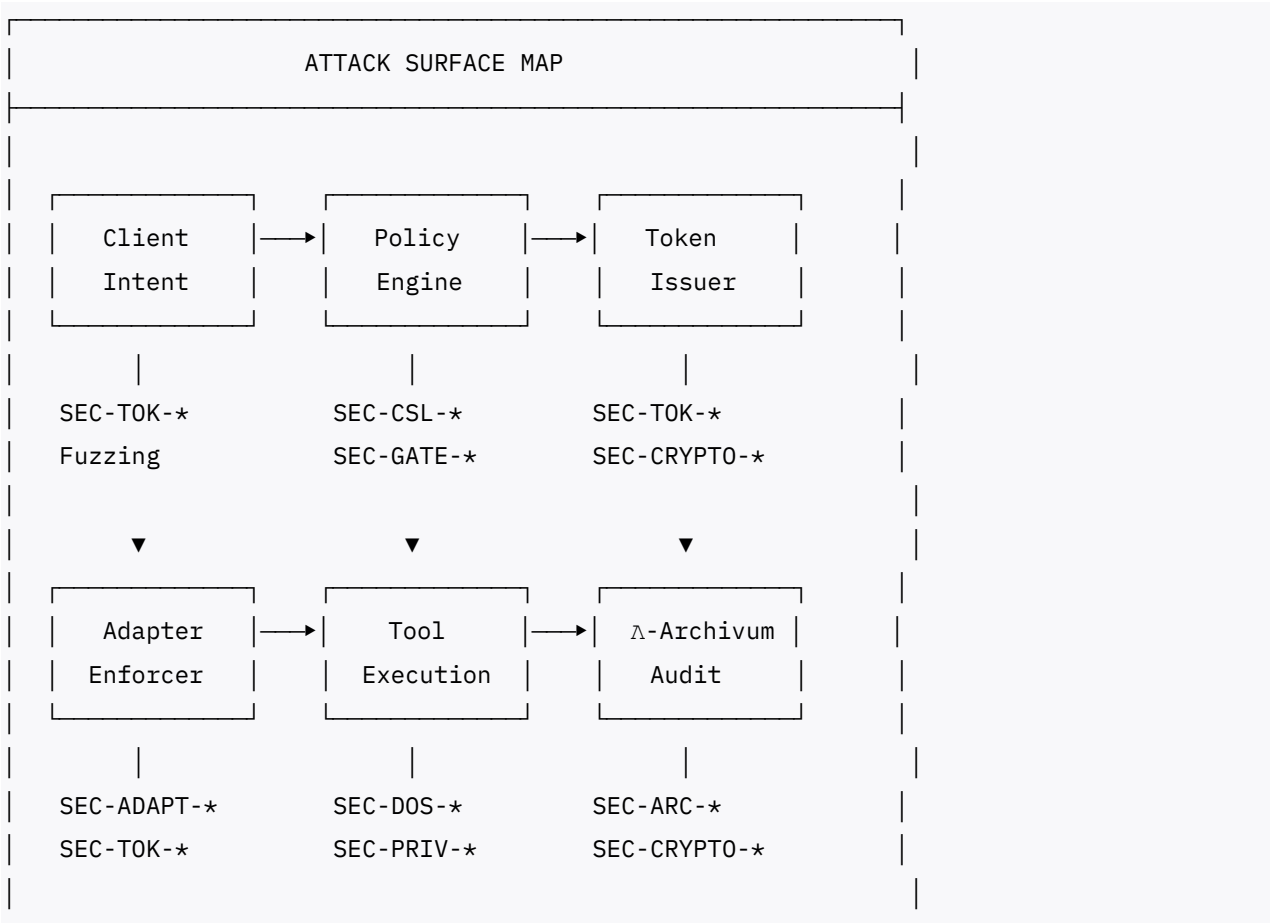
1. Establish prior art for adversarial testing patterns in capability-based authorization systems
2. Prevent patent enclosure of security testing methodologies for CSL-compliant systems
3. Enable interoperability of security validation across AProof implementations
4. Block trivial patent claims on token attack detection, CSL bypass prevention, and audit integrity verification

[\[4\]\[1\]](#)

Appendix A: Test ID Registry

Range	Category	Description
SEC-TOK-001 to 050	Token Attacks	Capability token manipulation
SEC-CSL-001 to 030	CSL Bypass	Ethical operator circumvention
SEC-GATE-001 to 025	Prime Gate	Prime-gated activation exploits
SEC-ARC-001 to 035	Audit Tampering	Λ -Archivum integrity attacks
SEC-ADAPT-001 to 030	Adapter Bypass	Enforcement boundary attacks
SEC-CRYPTO-001 to 025	Cryptographic	Implementation vulnerabilities
SEC-DOS-001 to 020	Denial of Service	Availability attacks
SEC-PRIV-001 to 020	Privacy	Information disclosure attacks

Appendix B: Attack Surface Coverage



END OF SECURITY TEST SUITE SPECIFICATION

This specification provides the adversarial testing framework necessary to validate that Λ Proof implementations resist active attackers, not merely function correctly under benign conditions. All implementations seeking Tier 2 or Tier 3 certification must pass the complete security test suite and undergo independent security audit.^[1]

**

-
1. [lets-produce-a-list-of-documen-MocV13I6RI.gUHQs0MYElw.md](#)
 2. Adapter-Coordination-Interface-Specification.pdf
 3. The-LProof-Architecture_Web4.pdf
 4. lets-discuss-licensing-open-so-HV_bbJ5uQIOPuXCF7b61YQ.md
 5. How Trust is Measured.pdf
 6. Λ Proof Token Issuance.pdf
 7. Audit Logs_ Event Schema & Sanitized Examples.pdf
 8. LProof-Intent-Schema-Specification.pdf
 9. Conscious Sovereignty Layer (CSL) Solver.pdf
 10. Λ Proof Protocols.pdf
 11. DAO Charter.pdf
 12. Building a Λ Dapp_ A Developer's Implementation Guide.pdf
 13. Capability Token Semantic Mapping.pdf
 14. Data Sovereignty Informational Personhood.pdf

15. Λ Proof_ An Architectural Blueprint for Verifiable Artificial Intelligence.pdf

16. Λ^P -Archivum.pdf

17. Governance for a Non-Surveillance Digital Ecosystem.pdf

18. Token Specification.pdf

19. Risk Classification.pdf

20. Policy Rules Specification.pdf

21. Evidence Validation.pdf